

Project Report

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY GUWAHATI

Department of Electronics and Communication Engineering

Smart Irrigation System using ESP32 and IoT

Student : Mallena Vardhan, Marakattu Purushottam
Roll No. : 2301123, 2301125
Programme : B.Tech in Electronics and Communication Engineering
Group : EC32
Semester : VI
Course Instructor : Dr. Sheikh Wasmir Hussain



Academic Year: 2025–2026

1 Objective

The objective of this project is to design and implement a smart irrigation system using ESP32 that automates water supply based on soil moisture and weather forecast data.

2 Abstract

This project presents a smart irrigation system using ESP32 and IoT technologies to automate water supply based on environmental conditions. The system integrates a DHT22 sensor for temperature and humidity measurement, a capacitive soil moisture sensor for soil condition monitoring, and weather forecast data obtained through the OpenWeather API.

The collected data is processed and uploaded to the ThingSpeak cloud platform, enabling real-time monitoring through a web dashboard. Based on soil moisture levels and rain probability, the system automatically controls a water pump using a relay module.

The proposed system helps in efficient water management by reducing unnecessary irrigation and ensuring optimal soil conditions. The results demonstrate reliable performance under different environmental conditions, making it suitable for smart agriculture applications.

3 Components Used

- **Microcontroller:** ESP32
- **Sensor:** DHT22 Temperature and Humidity Sensor
- **Soil Moisture Sensor:** Capacitive
- **Cloud Platform:** ThingSpeak
- **Weather API:** OpenWeather API
- **Programming Languages:** Embedded C, Python, HTML/JavaScript

4 System Level Design

The system is divided into three main modules:

- **Data Acquisition Module:** ESP32 collects data from DHT22 and soil moisture sensor.

- **Cloud Processing Module:** Weather data is fetched using OpenWeather API and stored in ThingSpeak.
- **Decision Module:** Based on sensor and weather data, irrigation decision is taken.

5 Sensors and Subsystems

5.1 DHT22 Sensor

The DHT22 sensor is used to measure temperature and humidity. It provides digital output with high accuracy and is widely used in IoT applications.

5.2 Soil Moisture Sensor

The capacitive soil moisture sensor measures the water content in soil by detecting changes in capacitance. Higher values indicate dry soil, while lower values indicate wet soil.

5.3 ESP32 Microcontroller

ESP32 is a powerful microcontroller with built-in Wi-Fi, used for data processing, communication, and control operations.

5.4 ThingSpeak Cloud

ThingSpeak is used to store and visualize sensor and weather data in real time.

5.5 Web Dashboard

A web interface developed using HTML and JavaScript is used to display system data and irrigation status.

6 Circuit Diagram

The circuit diagram of the smart irrigation system is shown in Fig. 1. It illustrates the interfacing of the ESP32 microcontroller with the DHT22 temperature-humidity sensor and the capacitive soil moisture sensor.

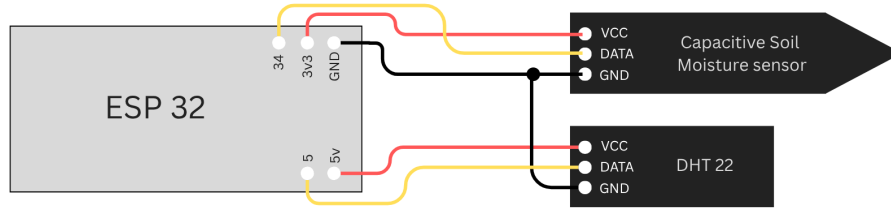


Figure 1: Interfacing of ESP32 with DHT22 and Soil Moisture Sensor

7 Implementation Steps

1. Connect DHT22 and soil moisture sensor to ESP32.
2. Establish Wi-Fi connection.
3. Configure ThingSpeak channels.
4. Run Python script to fetch weather data.
5. Upload ESP32 code.
6. Develop and open web dashboard.
7. Monitor system and verify irrigation decisions.

7.1 Experimental Setup

The experimental setup of the smart irrigation system is shown in Fig. 2. The system consists of an ESP32 microcontroller interfaced with a DHT22 temperature-humidity sensor and a capacitive soil moisture sensor.

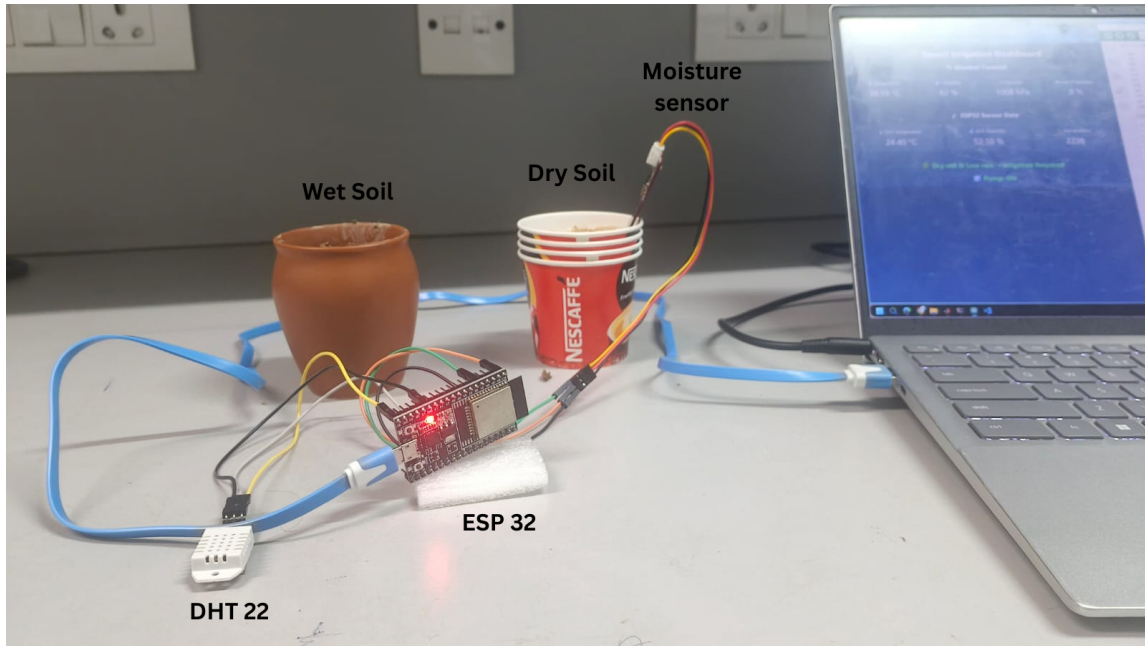


Figure 2: Experimental setup of the Smart Irrigation System

In this setup, the soil moisture sensor is inserted into two different soil conditions to demonstrate system behavior:

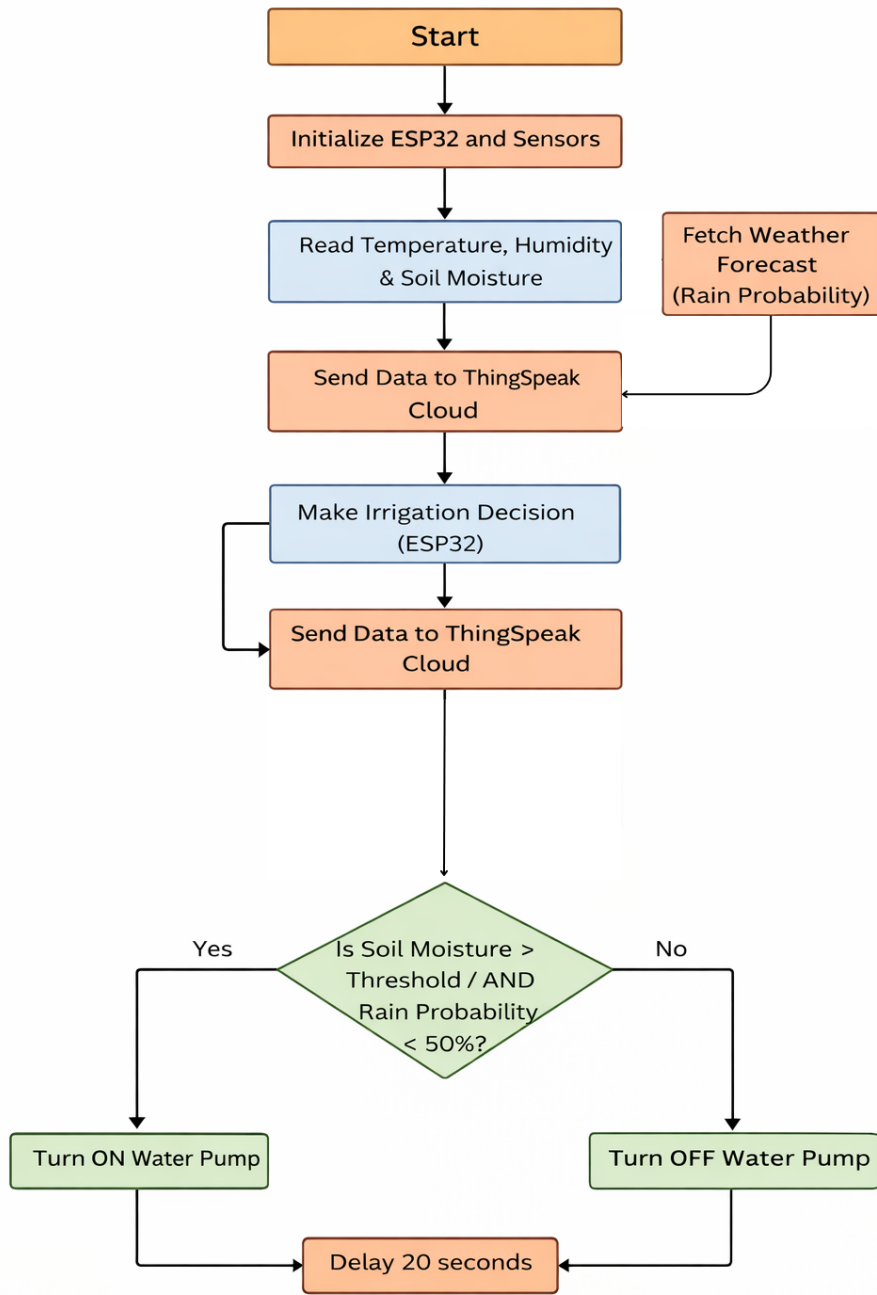
- **Wet Soil:** Represents sufficient moisture condition.
- **Dry Soil:** Represents low moisture condition requiring irrigation.

The ESP32 is connected to a laptop via USB for power supply and serial monitoring. The DHT22 sensor is used to measure ambient temperature and humidity, while the soil moisture sensor provides real-time soil condition data.

The system continuously reads sensor values and processes them along with weather data obtained from the cloud. Based on these inputs, it determines whether irrigation is required and displays the result on the serial monitor and web dashboard.

This setup validates the working of the system under practical conditions by comparing responses for both dry and wet soil environments.

8 Flowchart



System Level Design using ESP32

Figure 3: Flowchart of Smart Irrigation System

9 Algorithm

The proposed smart irrigation system follows an automated decision-making process based on real-time sensor data and weather forecast information. The ESP32 contin-

uously monitors soil moisture and environmental parameters while also considering rain probability obtained from the cloud. Based on these inputs, the system determines whether irrigation is required and controls the water pump accordingly.

Algorithm Steps

1. Initialize the ESP32 microcontroller, sensors (DHT22 and soil moisture), and Wi-Fi module.
2. Establish connection with the Wi-Fi network.
3. Connect to the ThingSpeak cloud platform.
4. Read temperature and humidity values from the DHT22 sensor.
5. Read soil moisture value from the analog sensor.
6. Fetch the latest rain probability data from ThingSpeak (weather channel).
7. Compare the obtained values using decision logic:
 - If (Soil Moisture > Threshold) AND (Rain Probability < 50%), then:
 - Turn ON the water pump (activate relay).
 - Else:
 - Turn OFF the water pump.
8. Upload the current sensor data (temperature, humidity, soil moisture) to ThingSpeak.
9. Wait for a fixed interval (20 seconds).
10. Repeat the process continuously.

10 Program Code

10.1 ESP32 Code

```
1 #include <WiFi.h>
2 #include "DHT.h"
3 #include <HttpClient.h>
4
5 // WiFi Credentials
6 const char* ssid = "Neo_Mallena_7";
```

```

7  const char* password = "947606139";
8
9  // ThingSpeak Configuration
10 String apiKey = "UDIWCD1PD07G04M9";
11 const char* server = "http://api.thingspeak.com/update";
12
13 // Sensor Configuration
14 #define DHTPIN 5
15 #define DHTTYPE DHT22
16 DHT dht(DHTPIN, DHTTYPE);
17
18 #define SOIL_PIN 34      // Analog input from soil moisture sensor
19 #define RELAY_PIN 4      // Controls water pump
20
21 // Global Variables
22 float temperature, humidity;
23 int soilValue;
24
25 // Function: Get Rain Probability
26 // Fetches rain probability from ThingSpeak (Field 4)
27 float getRainProbability() {
28     HTTPClient http;
29     String url = "http://api.thingspeak.com/channels/3319454/fields
        /4/last.txt";
30
31     http.begin(url);
32     int httpCode = http.GET();
33
34     float rain = 0;
35
36     if (httpCode > 0) {
37         rain = http.getString().toFloat();
38         Serial.print("Rain Probability: ");
39         Serial.println(rain);
40     } else {
41         Serial.println("Failed to fetch rain data");
42     }
43
44     http.end();
45     return rain;
46 }

```

```

47
48 void setup() {
49     Serial.begin(115200);
50     dht.begin();
51
52     pinMode(RELAY_PIN, OUTPUT);
53     digitalWrite(RELAY_PIN, LOW); // Pump OFF initially
54
55     // Connect to WiFi
56     WiFi.begin(ssid, password);
57     Serial.print("Connecting to WiFi");
58
59     while (WiFi.status() != WL_CONNECTED) {
60         delay(500);
61         Serial.print(".");
62     }
63
64     Serial.println("\nWiFi Connected");
65 }
66
67 void loop() {
68
69     // Reconnect WiFi if disconnected
70     if (WiFi.status() != WL_CONNECTED) {
71         Serial.println("Reconnecting WiFi...");
72         WiFi.begin(ssid, password);
73         delay(5000);
74         return;
75     }
76
77     // Read sensor data
78     temperature = dht.readTemperature();
79     humidity = dht.readHumidity();
80     soilValue = analogRead(SOIL_PIN);
81
82     if (isnan(temperature) || isnan(humidity)) {
83         Serial.println("DHT sensor read error");
84         return;
85     }
86
87     // Display sensor readings

```

```

88 Serial.println("----- Sensor Data -----");
89 Serial.printf("Temp: %.2f C \n", temperature);
90 Serial.printf("Humidity: %.2f %%\n", humidity);
91 Serial.printf("Soil: %d\n", soilValue);
92
93 // Get rain probability from cloud
94 float rainProb = getRainProbability();
95
96 // Decision Logic
97 int soilThreshold = 2000;
98
99 Serial.println("----- Decision -----");
100
101 if (soilValue > soilThreshold && rainProb < 50) {
102     digitalWrite(RELAY_PIN, HIGH);
103     Serial.println("Pump ON (Dry soil & low rain chance)");
104 } else {
105     digitalWrite(RELAY_PIN, LOW);
106     Serial.println("Pump OFF (Moist soil or rain expected)");
107 }
108
109 // Send Data to ThingSpeak
110 HTTPClient http;
111
112 String url = String(server) + "?api_key=" + apiKey +
113             "&field1=" + String(temperature) +
114             "&field2=" + String(humidity) +
115             "&field3=" + String(soilValue);
116
117 http.begin(url);
118 int httpResponseCode = http.GET();
119
120 if (httpResponseCode > 0) {
121     Serial.println("Data uploaded successfully");
122 } else {
123     Serial.println("Error uploading data");
124 }
125
126 http.end();
127
128 Serial.println("-----\n");

```

```

129
130     delay(20000); // ThingSpeak requires ~15 sec minimum delay
131 }

```

10.2 Python API Code

```

1 import requests
2 import time
3
4 # API CONFIGURATION
5 API_KEY = "12079324b376b2c7b67b7e320526cb28" # OpenWeatherMap
6     API key
7
8 TS_API_KEY = "TX4VVKKL21MLDSXF" # ThingSpeak Write
9     API key
10
11 CITY = "Guwahati"
12
13 while True:
14     try:
15         # Fetch Weather Data
16         url = f"https://api.openweathermap.org/data/2.5/forecast?
17             q={CITY}&appid={API_KEY}&units=metric"
18         response = requests.get(url)
19         data = response.json()
20
21         if "list" in data:
22             # Take the nearest forecast data point
23             forecast = data['list'][0]
24
25             temp = forecast['main']['temp']
26             humidity = forecast['main']['humidity']
27             pressure = forecast['main']['pressure']
28             rain_prob = forecast.get('pop', 0) # Probability of
29                 precipitation
30
31             rain_percent = round(rain_prob * 100, 2)
32
33             # ----- Display Data -----
34             print("----- Weather Data -----")
35             print(f"Temp: {temp} C ")
36             print(f"Humidity: {humidity} %")

```

```

32     print(f"Pressure: {pressure} hPa")
33     print(f"Rain Probability: {rain_percent}%")
34
35     # ----- Send Data to ThingSpeak -----
36     ts_url = (
37         f"https://api.thingspeak.com/update?api_key={
38             TS_API_KEY}"
39         f"&field1={temp}&field2={humidity}&field3={
40             pressure}&field4={rain_percent}"
41     )
42
43     ts_response = requests.get(ts_url)
44
45     if ts_response.status_code == 200:
46         print("Data sent to ThingSpeak!")
47     else:
48         print("Failed to send data")
49
50     else:
51         print("API Error:", data)
52
53     except Exception as e:
54         print("Error:", e)
55
56     print("-----\n")
57
58     # ThingSpeak requires minimum ~15 sec delay
59     time.sleep(20)

```

10.3 Web Dashboard Code

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <title>Smart Irrigation Dashboard</title>
6 <meta name="viewport" content="width=device-width, initial-scale
7     =1.0">
8
9 <style>
10 /* Page Styling */

```

```

10 body {
11     margin: 0;
12     font-family: 'Segoe UI', sans-serif;
13     background: linear-gradient(135deg, #1e3c72, #2a5298);
14     color: white;
15     text-align: center;
16 }
17
18 h1 {
19     margin: 20px 0;
20 }
21
22 /* Section Titles */
23 .section-title {
24     margin-top: 20px;
25     font-size: 1.4em;
26     font-weight: bold;
27 }
28
29 /* Grid Layout */
30 .container {
31     display: grid;
32     grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
33     gap: 15px;
34     padding: 20px;
35 }
36
37 /* Card UI */
38 .card {
39     background: rgba(255,255,255,0.1);
40     border-radius: 18px;
41     padding: 20px;
42     backdrop-filter: blur(10px);
43     transition: 0.3s;
44 }
45
46 .card:hover {
47     transform: scale(1.05);
48     background: rgba(255,255,255,0.2);
49 }
50

```

```

51 .value {
52     font-size: 1.8em;
53     margin-top: 10px;
54 }
55
56 /* Status Text */
57 .status {
58     margin: 15px;
59     font-size: 1.3em;
60     font-weight: bold;
61 }
62
63 .on { color: #00ff9d; }
64 .off { color: #ff4d4d; }
65 </style>
66 </head>
67
68 <body>
69
70 <h1>          Smart Irrigation Dashboard</h1>
71
72 <!-- ----- WEATHER DATA ----- -->
73 <div class="section-title">          Weather Forecast</div>
74 <div class="container">
75     <div class="card">
76         <div>Temperature</div>
77         <div id="w_temp" class="value">--</div>
78     </div>
79     <div class="card">
80         <div>Humidity</div>
81         <div id="w_hum" class="value">--</div>
82     </div>
83     <div class="card">
84         <div>Pressure</div>
85         <div id="w_pres" class="value">--</div>
86     </div>
87     <div class="card">
88         <div>Rain Probability</div>
89         <div id="w_rain" class="value">--</div>
90     </div>
91 </div>

```

```

92
93 <!-- ----- ESP32 SENSOR DATA ----- -->
94 <div class="section-title">          Sensor Data</div>
95 <div class="container">
96     <div class="card">
97         <div>DHT Temperature</div>
98         <div id="s_temp" class="value">--</div>
99     </div>
100    <div class="card">
101        <div>DHT Humidity</div>
102        <div id="s_hum" class="value">--</div>
103    </div>
104    <div class="card">
105        <div>Soil Moisture</div>
106        <div id="soil" class="value">--</div>
107    </div>
108 </div>
109
110 <!-- ----- DECISION OUTPUT ----- -->
111 <div class="status" id="decision">Analyzing...</div>
112 <div class="status" id="pumpStatus">Pump: --</div>
113
114 <script>
115
116 // ThingSpeak Channels
117 const weatherChannel = "3319454";
118 const espChannel = "3329419";
119
120 // API URLs (latest entry)
121 const weatherURL = 'https://api.thingspeak.com/channels/${
122     weatherChannel}/feeds.json?results=1';
123
124 const espURL = 'https://api.thingspeak.com/channels/${espChannel
125     }/feeds.json?results=1';
126
127 // Fetch Data from Cloud
128 async function fetchData() {
129     try {
130         const weatherRes = await fetch(weatherURL);
131         const weatherData = await weatherRes.json();
132
133         const espRes = await fetch(espURL);

```

```

131     const espData = await espRes.json();
132
133     let w = weatherData.feeds[0];
134     let s = espData.feeds[0];
135
136     // Update Weather UI
137     document.getElementById("w_temp").innerText = w.field1 +
138         " C ";
139     document.getElementById("w_hum").innerText = w.field2 +
140         " %";
141     document.getElementById("w_pres").innerText = w.field3 +
142         " hPa";
143     document.getElementById("w_rain").innerText = w.field4 +
144         " %";
145
146     // Update Sensor UI
147     document.getElementById("s_temp").innerText = s.field1 +
148         " C ";
149     document.getElementById("s_hum").innerText = s.field2 +
150         " %";
151     document.getElementById("soil").innerText = s.field3;
152
153     // Convert values for logic
154     let rainProb = parseFloat(w.field4);
155     let soil = parseInt(s.field3);
156
157     updateDecision(rainProb, soil);
158
159 } catch (error) {
160     console.log("Error fetching data:", error);
161 }
162
163 // Decision Logic
164 function updateDecision(rainProb, soil) {
165
166     if (isNaN(rainProb) || isNaN(soil)) return;
167
168     let decisionText = document.getElementById("decision");
169     let pumpText = document.getElementById("pumpStatus");

```

```

166     let soilThreshold = 2000;
167
168     if (soil > soilThreshold && rainProb < 50) {
169         decisionText.innerHTML = "          Dry soil & low rain
170             Irrigation needed";
171         decisionText.className = "status on";
172
173         pumpText.innerHTML = "          Pump: ON";
174         pumpText.className = "status on";
175     } else {
176         decisionText.innerHTML = "          Moist soil or rain
177             expected          No irrigation";
178         decisionText.className = "status off";
179
180         pumpText.innerHTML = "          Pump: OFF";
181         pumpText.className = "status off";
182     }
183 }
184
185 // Auto Refresh
186 fetchData();
187 setInterval(fetchData, 20000); // 20 sec refresh (ThingSpeak safe
188 )
189
190 </script>
191
192 </body>
193 </html>

```

11 Results

The developed smart irrigation system was successfully tested under different environmental conditions. The system continuously monitored sensor data and made real-time irrigation decisions based on soil moisture and rain probability.

Sensor Data Output

The ESP32 reads temperature, humidity, and soil moisture values and displays them on the serial monitor. The system was observed to provide stable and consistent readings during operation.

```

11:55:32.734 -> Pump ON (Dry soil + No rain)
11:55:33.553 -> Data sent to ThingSpeak!
11:55:33.553 -> -----
11:55:33.553 -> ----- Sensor Data -----
11:55:33.542 -> Temp: 24.40
11:55:33.542 -> Humidity: 51.90
11:55:33.542 -> Soil: 2238
11:55:34.640 -> Rain Probability: 0.00
11:55:34.640 -> ----- Decision -----
11:55:34.640 -> Pump ON (Dry soil + No rain)
11:55:35.463 -> Data sent to ThingSpeak!
11:55:35.463 -> -----
11:55:35.463 -> ----- Sensor Data -----
11:56:15.443 -> Temp: 24.40
11:56:15.443 -> Humidity: 51.80
11:56:15.443 -> Soil: 2240
11:56:16.361 -> Rain Probability: 0.00
11:56:16.361 -> ----- Decision -----
11:56:16.361 -> Pump ON (Dry soil + No rain)
11:56:17.157 -> Data sent to ThingSpeak!
11:56:17.157 -> -----
11:56:17.157 -> ----- Sensor Data -----
11:56:37.179 -> Temp: 24.40
11:56:37.179 -> Humidity: 52.10
11:56:37.179 -> Soil: 2238

```

Figure 4: Serial Monitor Output showing sensor readings and pump decision

Weather Data Integration

The Python script successfully fetched weather forecast data from the OpenWeather API and uploaded parameters such as temperature, humidity, pressure, and rain probability to ThingSpeak.

```

PS C:\Users\Mallena\PSoc_embedded_system\Project_Irrigation_system> &
python314/python.exe c:/Users/Mallena/PSoc_embedded_system/Project_Irr
on_system/API_test_1.py
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!
Temp: 24.95, Humidity: 83, Pressure: 1009
Chance of rain: 20.0%
Data sent to ThingSpeak!

```

Figure 5: Weather data fetched from API and sent to ThingSpeak

Web Dashboard Visualization

A real-time dashboard was developed to display both sensor data and weather data. The dashboard clearly shows environmental parameters and irrigation status.

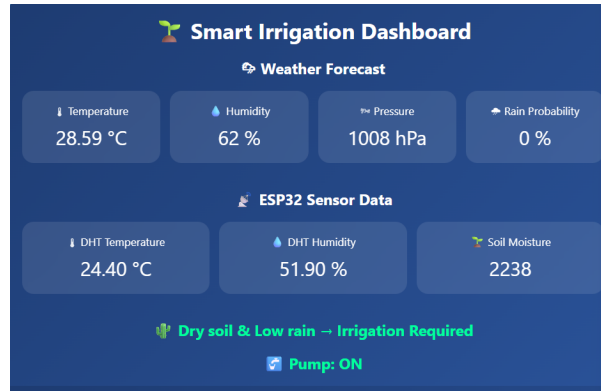


Figure 6: Smart Irrigation Web Dashboard

System Behavior

The system was tested under different conditions:

- When soil moisture value was high (dry soil) and rain probability was low, the pump was turned ON.
- When soil moisture was sufficient or rain probability was high, the pump remained OFF.

The results confirm that the system performs accurate decision-making and responds effectively to changing environmental conditions.

12 Conclusion

In this project, a smart irrigation system based on ESP32 was successfully designed and implemented. The system integrates real-time sensor data and weather forecast information to automate irrigation.

The use of IoT platforms such as ThingSpeak enables remote monitoring and data visualization. The system effectively reduces water wastage by ensuring irrigation only when required.

Here is the github repository link for the following project [Github link](#)

13 Future Scope

The system can be further enhanced by integrating mobile applications for remote control, adding machine learning algorithms for better irrigation prediction, and implementing solar-powered energy solutions. Multi-zone irrigation and advanced sensors can also be incorporated to improve efficiency and scalability.